

Microprocessor chosen: Any intel microprocessors that use the Intel 64 or IA-32 architectures. Examples include the AMD Athlon 64, Opteron, or Phenom as well as the Intel Pentium 4 and Core 2 Duo CPUs (i3, i5, i7, i9).

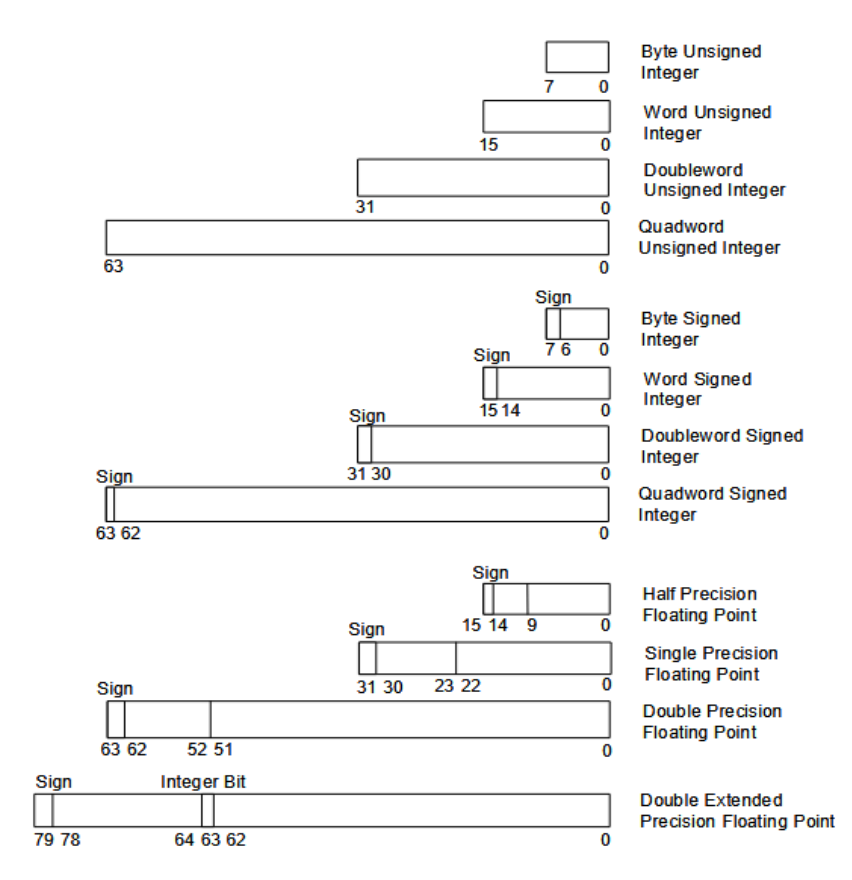
I used the [Intel® 64 and IA-32 Architectures Software Developer Manuals](#) as my main source of information.

Data types, Space, and Representation

The fundamental data types that the Intel IA-32 architecture operates on are as follows:

- **Byte** (8 bits)
- **Word** (2 bytes, 16 bits)
- **Doubleword** (4 bytes, 32 bits)
- **Quadword** (8 bytes, 64 bits)
- **Double Quadword** (16 bytes, 128 bits)

These are the lowest level data types that the CPU can operate on and comprise the numeric data types as shown in the table. Everything is relatively standard with unsigned integers being 1 byte and signed integers using Two's Complement representation.



Floating-point numbers

The IA-32 architecture follows the IEEE 754 standard for floating-point arithmetic and defines four distinct floating point data types:

- Half precision
- Single precision
- Double precision
- Double-extended precision

The length, precision, and range for these four types are shown in the table below.

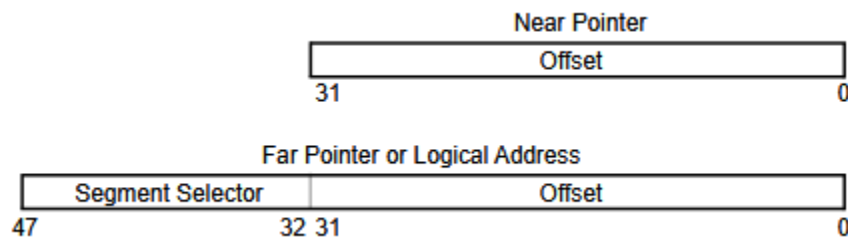
Data Type	Length (Bits)	Precision (Bits)	Approximate Normalized Range	
			Binary	Decimal
Half Precision	16	11	2^{-14} to 2^{16}	6.10×10^{-5} to 6.55×10^4
Single Precision	32	24	2^{-126} to 2^{128}	1.18×10^{-38} to 3.40×10^{38}
Double Precision	64	53	2^{-1022} to 2^{1024}	2.23×10^{-308} to 1.80×10^{308}
Double-Extended Precision	80	64	2^{-16382} to 2^{16384}	3.36×10^{-4932} to 1.19×10^{4932}

Strings

Intel defines strings as continuous sequences of bits, bytes, words, or doublewords. “A bit string can begin at any bit position of any byte and can contain up to $2^{32} - 1$ bits. A byte string can contain bytes, words, or doublewords and can range from zero to $2^{32} - 1$ bytes (4 GBytes).” (Intel Software Developer Handbook).

Pointers

There are two pointer types defined: near points and far pointers. A near pointer is a 32 or 16 bit offset within a segment. The reason for the absence of a segment selector is because the segment is implied by the CPU. This is commonly used within flat memory which only needs an offset since the segment is always known (implied). Far pointers, on the other hand, contain the offset as well as the segment. Far points can also come in either 32 or 16 bit sizes.



There are plenty more supported data types such as floating point numbers, real numbers, non-number encodings, normalized numbers, signed and unsigned zeros, NaNs, etc. Far too many to detail in this paper, but I have specified all the major and necessary data types above.

Tools for Multilanguage development in Virtual Studio Code

Virtual Studio Code (VS Code) is famous for its extensions and relatively liberal customization. If one wanted to localize their VS Code to simplified Chinese, all they would need to do is go to the extensions within the app and download the language pack. After which the user would be able to code and read in their local language. There are two types of language packs that a user can download and use in VS Code: official (Microsoft developed and supported) and unofficial (Private user/organization developed and supported). The following is a list of all **official** language packs that are offered by Microsoft:

- English (U.S, default)
- French
- Italian
- German
- Spanish
- Russian
- Chinese (simplified)
- Chinese (Traditional)
- Japanese
- Korean
- Czech
- Portuguese (Brazil)
- Turkish
- Polish
- Hungarian
- Bulgarian

Something to keep in mind is that unofficial language packs vary in quality of translation, maintenance, completeness of UI translation, and are less than ideal in comparison to official languages supported by Microsoft.

Internal representation for non-english languages

Effectively all officially supported languages in VS Code use Unicode for internal representation. This is because Unicode was made for the purpose of supporting every language. VS Code is built with the Electron framework, which uses a Chromium based browser to turn a web app into a local desktop app. The Electron framework is coded using the languages that are used for the web: HTML, CSS, and JavaScript. HTML's default language encoder is Unicode, which is why VS Code (among the previously mentioned reasons) uses that same encoder.

The Python Standard Library

Python provides a standard library (pre-written pieces of code that serve as utilities for the developer) that comes packed with many classes, data types, functions/methods, and modules. The following is a list of the most commonly used modules from the PSL.

- Sys (system manipulation and data access)
- OS (same as system but with the operating system)
- JSON
- CSV
- Datetime
- Math
- Random
- Threading (extremely useful)
- Multicore processing
- Asyncio
- HTTP
- Socket

As well as many others. Like I said, the PSL is massive. More detail can be found at [The Python Standard Library](#)

What Data Structures are Supported by the Python Standard Library

The PSL provides the developer with all the basic data structures such as

- Lists
- Tupels
- Dictionaries (hash lists)
- Sets

But where the PSL really shines is in the collections module which provides the developer with a plethora of more nuance data types that can be used for optimization or advanced algorithms. Some of the data structures provided within the collections module are

- **Deque**: double ended queue, allows for fast appends and popping (removing and returning the last value) from both ends.
- **Defaultdict**: regular dictionary except it provides a default value for missing keys.
- **Ordereddict**: regular dictionary except it preserves insertion order.
- **Chainmap**: groups multiple dictionaries into one view.
- **Lifoqueue**: Last in first out queue.

What Types of Data are Recommended for Use with Each Data Structure Object

Some examples of what type of real-words data would be used for the structures I specified above are:

Deque:

- Very useful for algorithms that require sliding-window functions such as real time stock price tracking or finding specific sub-arrays within an array (many use cases).

Defaultdict:

- Useful for when one has a dictionary that they know will have a value that needs to be grouped by a key, but that key might not exist. Instead of checking if the key exists every time they insert a

value, they can use a defaultdict. To solve this issue, a default dict will insert the value even if the key you specify doesn't exist. The default dict inserts the key with a default (previously defined) value. A real-world example may be grouping students by classes where you don't have all the classes in a data set, and only have student-class pairs.

Ordereddict

- This is simply an ordered dictionary based on insertion. A use case might be steps in configuring a program's settings, where each configuration must be applied in sequential order (as is commonplace in these scenarios).

Chainmap:

- Chainmaps are a data structure that groups dictionaries into a single group called a view. The best example of this data structure is CSS cascading and specificity. In the cascading process, different configurations are given different priority in a hierarchy. The same can be done with chainmaps, where certain views are triggered sequentially and depending on user preference or order of importance.

Which Classes Contain Which Data Structures

In Python, everything from integers and strings to complex user-created classes are classes. With that being said, here are a couple of classes from the PSL and what data structures they implement.

Lists:

- Lists implement dynamic arrays, a data structure that allows for efficient appendation and removal. One of the most useful features that dynamic arrays implement is that they grow with each element added, instead of having a fixed size like a static array (hence dynamic). Each element in a dynamic array is actually just a pointer to the address of the data pertaining to that element. For example, a list like [10, "hello", True] doesn't contain an integer, string, and boolean, but three pointers that point to the memory address that contain those values.

Dictionaries:

- Dictionaries are Python's implementation of hash maps, a data structure that maps key-value pairs. Similarly to lists (dynamic arrays), a dictionary such as dict = {"name": age} does not contain a string and an integer, but two pointers that point to the name and age variables that are stored elsewhere in memory. What connects these two values (pointers) is that they are stored in the same structure internally (The Python interpreter turns the key-value pairs into a C struct and stores it as such).